

A neural network-evolutionary computational framework for remaining useful life estimation of mechanical systems

David Laredo^a, Zhaoyin Chen^a, Oliver Schütze^{b,c}, Jian-Qiao Sun^{a,*}

^a Department of Mechanical Engineering, School of Engineering, University of California, Merced, CA 95343, USA

^b Department of Computer Science, CINVESTAV, Mexico City, Mexico

^c Rodolfo Quintero Chair, UAM Cuajimalpa, Mexico

ARTICLE INFO

Article history:

Received 18 October 2018

Received in revised form 18 April 2019

Accepted 23 April 2019

Available online 4 May 2019

Keywords:

Artificial neural networks

Moving time window

RUL estimation

Prognostics

Evolutionary algorithms

ABSTRACT

This paper presents a framework for estimating the remaining useful life (RUL) of mechanical systems. The framework consists of a multi-layer perceptron and an evolutionary algorithm for optimizing the data-related parameters. The framework makes use of a strided time window along with a piecewise linear model to estimate the RUL for each mechanical component. Tuning the data-related parameters in the optimization framework allows for the use of simple models, e.g. neural networks with few hidden layers and few neurons at each layer, which may be deployed in environments with limited resources such as embedded systems. The proposed method is evaluated on the publicly available C-MAPSS dataset. The accuracy of the proposed method is compared against other state-of-the-art methods in the literature. The proposed method is shown to perform better than the compared methods while making use of a compact model.

© 2019 Elsevier Ltd. All rights reserved.

1. Introduction

Traditionally, maintenance of mechanical systems has been carried out based on scheduling strategies. Such strategies are often costly and less capable of meeting the increasing demand of efficiency and reliability (Gebrael, Lawley, Liu, & Ryan, 2005; Zaidan, Mills, & Harrison, 2013). Condition based maintenance (CBM) also known as intelligent prognostics and health management (PHM) allows for maintenance based on the current health of the system, thus cutting down the costs and increasing the reliability of the system (Zhao, Bin, Wang, & Lu, 2017). Here, we refer to prognostics as the estimation of remaining useful life of a system. The remaining useful life (RUL) of the system can be estimated based on the historical data. This data-driven approach can help optimize maintenance schedules to avoid engineering failures and to save costs (Lee et al., 2014).

The existing PHM methods can be grouped into three different categories: model-based (Yu & Kuffi, 2001), data-driven (Liu & Wang, 2009; Mosallam, Medjaher, & Zerhouni, 2013) and hybrid approaches (Liu, Wang, & Yang, 2012; Pecht & Jaai, 2010). Model-based approaches attempt to incorporate physical models of the system into the estimation of the RUL. If the system degradation is modeled precisely, model-based approaches usually exhibit better performance than data-driven approaches

(Qian, Yan, & Gao, 2017). This comes at the expense of having extensive a priori knowledge of the underlying system and having a fine-grained model of the system, which can involve expensive computations. On the other hand, data-driven approaches use pattern recognition to detect changes in system states. Data-driven approaches are appropriate when the understanding of the first principles of the system dynamics is not comprehensive or when the system is sufficiently complex such as jet engines, car engines and complex machineries, for which it is prohibitively difficult to develop an accurate model.

Common disadvantages for the data-driven approaches are that they usually exhibit wider confidence intervals than model-based approaches and that a fair amount of data is required for training. Many data-driven algorithms have been proposed. Good prognostics results have been achieved. Among the most popular algorithms we can find artificial neural networks (ANNs) (Gebrael, Lawley, Li, & Parmeshwaran, 2004), support vector machine (SVM) (Benkedjouh, Medjaher, Zerhouni, & Rechak, 2013), Markov hidden chains (MHC) (Dong & He, 2007) and so on. Over the past few years, data-driven approaches have gained more attention in the PHM community. A number of machine learning techniques, especially neural networks, have been applied successfully to estimate the RUL of diverse mechanical systems. ANNs have demonstrated good performance in modeling highly nonlinear, complex, multi-dimensional systems without any prior knowledge on the system behavior (Li, Ding, & Sun, 2018). While

* Corresponding author.

E-mail address: jqsun@ucmerced.edu (J.-Q. Sun).

Table 1
C-MAPSS dataset details.

Dataset	C-MAPSS			
	FD001	FD002	FD003	FD004
Training trajectories	100	260	100	248
Test trajectories	100	259	100	248
Operating conditions	1	6	1	6
Fault modes	1	1	2	2

the confidence limits for the RUL predictions cannot be analytically provided (Sikorska, Hodkiewicz, & Ma, 2011), the neural network approaches are promising for prognostic problems.

Neural networks for estimating the RUL of jet engines have been previously explored in Lim, Goh, and Tan (2016) where the authors propose a multi-layer perceptron (MLP) coupled with a feature extraction (FE) method and a time window for the generation of the features for the MLP. In the publication, the authors demonstrate that a moving window combined with a suitable feature extractor can improve the RUL prediction as compared with the studies with other similar methods in the literature. In Li et al. (2018), the authors explore a deep learning ANN architecture, the so-called convolutional neural networks (CNNs), where they demonstrate that by using a CNN without any pooling layers coupled with a time window, the predicted RUL is further improved.

In this paper we propose a novel framework for estimating the RUL of complex mechanical systems. The framework consists of a MLP to estimate the RUL of the system, coupled with an evolutionary algorithm for the fine tuning of data-related parameters. In this paper we refer to data-related parameters as parameters that define the shape, defined in terms of window size and window stride, and quality of the data, measured with respect to some performance indicators, used by the MLP. Please note that while this specific framework makes use of a MLP, the framework can in principle use several other learning algorithms, our main objective is the treatment of the data instead of the choice of a particular learning algorithm. The publicly available NASA C-MAPSS dataset (Saxena & Goebel, 2008) is used to assess the efficiency and reliability of the proposed framework, by efficiency we mean the complexity of the used regressor and reliability refers to the accuracy of the predictions made by the regressor. This approach allows for a simple and small MLP to obtain better results than those reported in the current literature while using less computing power.

The remainder of this paper is organized as follows. The C-MAPSS dataset is presented in Section 2. The framework and its components are thoroughly reviewed in Section 3. The method is evaluated using the C-MAPSS dataset in Section 4. A comparison with the state-of-the-art is also provided. Finally, the conclusions are presented in Section 5.

2. NASA C-MAPSS dataset

The NASA C-MAPSS dataset is used to evaluate performance of the proposed method (Saxena & Goebel, 2008). The C-MAPSS dataset contains simulated data produced using a model based simulation program developed by NASA. The dataset is further divided into 4 subsets composed of multi-variate temporal data obtained from 21 sensors.

For each of the 4 subsets, a training and a test set are provided. The training sets include run-to-failure sensor records of multiple aero-engines collected under different operational conditions and fault modes as described in Table 1.

The data is arranged in an $n \times 26$ matrix where n is the number of data points in each subset. The first two variables

represent the engine and cycle numbers, respectively. The following three variables are operational settings which correspond to the conditions in Table 1 and have a substantial effect on the engine performance. The remaining variables represent the 21 sensor readings that contain the information about the engine degradation over time.

Each trajectory within the training and test sets represents the life cycles of the engine. Each engine is simulated with different initial health conditions, i.e. no initial faults. For each trajectory of an engine the last data entry corresponds to the cycle at which the engine is found faulty. On the other hand, the trajectories of the test sets terminate at some point prior to failure, hence the need to predict the remaining useful life. The aim of the MLP model is to predict the RUL of each engine in the test set. The actual RUL values of test trajectories are also included in the dataset for verification. Further discussions of the dataset and details on how the data is generated can be found in Saxena, Goebel, Simon, and Eklund (2008).

2.1. Performance metrics

To evaluate the performance of the proposed approach on the C-MAPSS dataset, we make use of two scoring indicators, namely the Root Mean Squared Error (RMSE) denoted as $e_{rms}(d)$ and a score proposed in Saxena et al. (2008) which we refer as the RUL Health Score (RHS) denoted as $s_{rh}(d)$. The two scores are defined as follows,

$$e_{rms} = \sqrt{\frac{1}{N} \sum_{i=1}^N d_i^2} \quad (1)$$

$$s_{rh} = \frac{1}{N} \sum_{i=1}^N s_i \quad (2)$$

$$s_i = \begin{cases} e^{-\frac{d_i}{13}} - 1, & d_i < 0 \\ e^{\frac{d_i}{10}} - 1, & d_i \geq 0 \end{cases}$$

where N is the total number of samples in the test set and $d = \hat{y} - y$ is the error between the estimated RUL values $\hat{y}$, and the actual RUL values y for each engine within the test set. It is important to note that $s_{rh}(d)$ penalizes late predictions more than early predictions since usually late predictions lead to more severe consequences in fields such as aerospace.

3. Framework description

In this section, the proposed ANN-EA based method for prognostics is presented. The method makes use of a multi-layer perceptron (MLP) as the main regressor for estimating the RUL of the engines in the C-MAPSS dataset. The choice of a MLP as the learning algorithm instead of any of the other choices (SVM, RNN, CNN, Least-Squares, etc.) obeys the fact that MLPs are in general good for nonlinear data like the one exhibited by the C-MAPSS dataset, but at the same time are less computationally expensive than some of the more sophisticated algorithms as the CNN or the RNN. Indeed, the RNN may be a more suitable choice for this particular problem since it involves time-sequenced data, nevertheless, we will show that by doing a fine tuning of the data-related parameters (and thus data processing), the inference power of a simple MLP can be competitive even when compared against that of an RNN. For the training sets, the feature vectors are generated by using a moving time window while a label vector is generated with the RUL of the engine. The label has a constant RUL for the early cycles of the simulation, and becomes a linearly decreasing function of the cycle in the remaining cycles.

Table 2
Results for different architectures for subset 1, 100 epochs.

Tested architecture	RMSE				RHS			
	min.	max.	avg.	STD	min.	max.	avg.	STD
Architecture 1	15.51	17.15	16.22	0.49	4.60	7.66	5.98	0.91
Architecture 2	15.24	16.46	15.87	0.47	4.07	6.26	5.29	0.82
Architecture 3	15.77	17.27	16.15	0.45	5.11	8.25	5.93	0.94
Architecture 4	15.13	17.01	15.97	0.47	3.90	7.54	5.65	1.2
Architecture 5	16.39	17.14	16.81	0.23	5.19	6.58	5.98	0.42
Architecture 6	16.42	17.36	16.87	0.30	5.15	7.09	6.12	0.62

This is the so-called piecewise linear degradation model (Ramasso, 2014). For the test set, a time window is taken from the last sensor readings of the engine. The data of the test set is used to predict the RUL of the engine.

The window-size n_w , window-stride n_s , and early-RUL R_e are data-related parameters, which for the sake of clarity and formalism in this study, form a vector $v \in \mathbb{Z}^3$ such that $v = (n_w, n_s, R_e)$. The vector v has a considerable impact on the quality of the predictions by the regressor. It is computationally intensive to find the best parameters of v given the search space inherent to these parameters. In this paper, we propose an evolutionary algorithm to optimize the data-related parameters v . The optimized parameter set v allows the use of a simple neural network architecture while attaining better results in terms of the quality of the predictions compared with the results by other methods in the literature.

3.1. The network architecture

After careful examinations of the C-MAPSS dataset, we propose to use a rather simple MLP architecture for all the four subsets of the data. The choice of a simple architecture over a more complex one follows the fact that simpler neural networks are less computationally expensive to train, furthermore, the inference is done also faster since less operations are involved. To measure the simplicity/complexity of a neural network we use the number of trainable parameters (weights) of the neural network, usually the more trainable parameters in a network the more computations that need be done, thus increasing the computational burden of the training/inference process. The implementations are done in Python using the Keras/Tensorflow environment. The source code is publicly available at the git repository https://github.com/dlaredo/NASA_RUL-CMAPS (Laredo, Chen, Schütze, & Sun, 2018).

The choice of the network architecture is made by following an iterative process, our goal was to find a good compromise between efficiency (simple neural network models) and reliability (scores obtained by the model using the RMSE metric): We compared 6 different architectures (see Appendix A), training each for 100 epochs using a mini-batch size of 512 and averaging their results over 10 different runs. L1 (Lasso) and L2 regularization (Ridge) (Bühlmann & Geer, 2011) are used to prevent overfitting. L1 regularization penalizes the sum of the absolute value of the weights and biases of the networks, while L2 regularization penalizes the sum of the squared value of the weights and biases. Two objectives are pursued during the iterations: (1) the architecture must be minimal in terms of the number of trainable parameters; and (2) the performance indicators must be minimized.

The process for choosing the network architecture is as follows. First, choose a v for the experiment, say $v = (30, 1, 140)$. Next, six different ANN architectures are defined. These architectures are provided in the Appendix A. For each of the six different architectures, its performance is assessed using a cross-validation

Table 3
Proposed neural network architecture.

Layer	Neurons	Activation	Additional information
Fully connected	20	ReLU	$L1 = 0.1, L2 = 0.2$
Fully connected	20	ReLU	$L1 = 0.1, L2 = 0.2$
Fully connected	1	Linear	$L1 = 0.1, L2 = 0.2$

set from subset 1 of C-MAPSS. Table 2 summarizes the results for each tested architecture, while Table 3 presents the architecture chosen for the remainder of this work (which provides the best compromise between compactness and performance among the tested architectures). Each row in the table represents a neural network layer while each column describes each one of the key parameters of the layer such as the type of layer, number of neurons in the layer, activation function of the layer and whether regularization is used, where L1 denotes the L1 regularization factor and L2 denotes the L2 regularization factor, the order in which the layers are appended from the table is top-bottom. Definitions for the used activation functions are presented in Appendix B.

3.2. Shaping the data

This section covers the data preprocessing applied to the raw sensor readings in each of the datasets. Although the original datasets contain 21 different sensor readings, some of the sensors do not present much variance or convey redundant information, our choice of the sensors is based on previous studies such as Li et al. (2018), Lim et al. (2016) where it was discovered that some sensor values do not vary at all throughout the entire engine life cycle while some others are redundant according to PCA or clustering analysis. These sensors are therefore discarded. In the end, only 14 sensor readings out of the 21 are considered for this study. Their indices are $\{2, 3, 4, 7, 8, 9, 11, 12, 13, 14, 15, 17, 20, 21\}$. The raw measurements are then used to create the strided time windows with window-size n_w and window-stride n_s . For the training labels, R_e is used at the early stages and then the RUL is linearly decreased. The data is normalized to be within the range $[-1, 1]$ using the min-max normalization.

$$\hat{x}_i = 2 * \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)} - 1, \quad (3)$$

where x_i denotes the m -dimensional vector whose components are all the readings for the i -th sensor and $\hat{x}_i$ is the normalized x_i vector.

3.2.1. Time window and stride

In multivariate time-series problems such as RUL, more information can be generally obtained from the temporal sequence of the data as compared with the multivariate data point at a single time stamp. For a time window of size n_w with a stride $n_s = 1$, all the sensor readings in the time window form a feature vector $\mathbf{x} \in \mathbb{R}^{s \times n_w}$, where s denotes the number of sensors being read. Stacking together m of this time window forms feature vector $\mathbf{X} \in \mathbb{R}^{m \times s \times n_w}$ while its corresponding RUL values are defined as $y \in \mathbb{Z}^m$. It is important to mention that the shape of $\mathbf{X}$ defines the number of input neurons for the neural network, therefore changing the shape of $\mathbf{X}$ effectively changes the number of inputs to the neural network. This approach has successfully been tested in Li et al. (2018), Lim et al. (2016) where the authors propose the use of a moving window with sizes ranging from 20 to 30. In this paper, we propose not only the use of a moving time window, but also a *strided* time window that updates more than one element ($n_s > 1$) at the time. A graphical depiction of the strided time

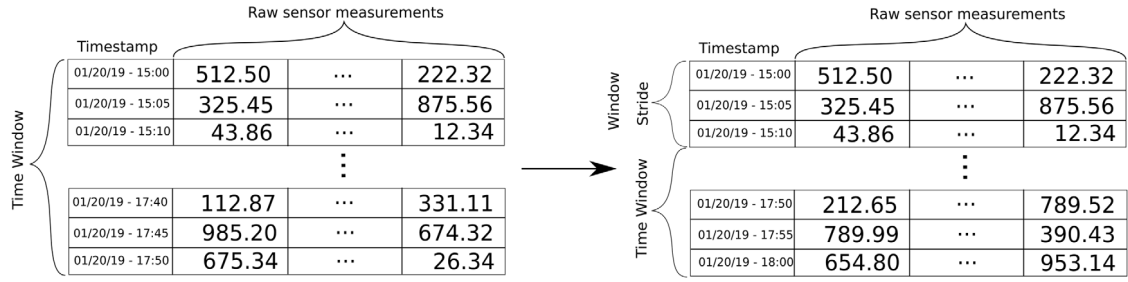


Fig. 1. Graphical depiction of the time window used in this framework.

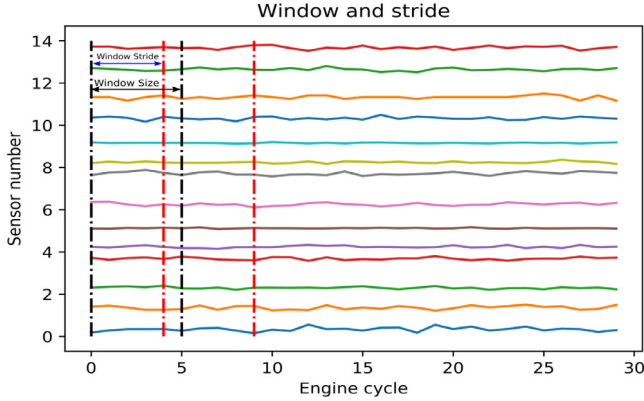


Fig. 2. Window size and stride example.

window is shown in Fig. 1. For Fig. 1 the numbers and time-stamps are just illustrative, the window size exemplified is of 30 time-stamps while the stride is of 3 time-stamps.

Fig. 2 shows an example of how to form a sample vector, in this example $s = 14$, $n_w = 5$ and $n_s = 4$. Each one of the plotted lines denotes the readings for each of the fourteen chosen sensors. The dashed vertical lines (black lines) represent the size of the window, in this case we depict a window size of 5 cycles. For the next window (red dashed lines) the time window is advanced by a stride of 4 cycles, note that some sensor readings may be overlapped for different moving windows. For every window, the sensor readings are appended one after another to form a vector of 14×5 features, for this specific case the unrolled vector will be of 70 features.

The use of a strided time window allows for the regressor to take advantage not only of the previous information, but also to control the ratio at which the algorithm is fed with new information. With the usual time window approach, only one point is updated for every new time window. The strided time window considered in this study allows for updating more than one point at the time for the algorithm to make use of the new information with less iterations. Our choice of the strided time window is inspired by the use of strided convolutions in Convolutional Neural Networks (Kong & Lucey, 2017). Further studies of the impact of the stride on the prediction should be done in the future.

3.2.2. Piecewise linear degradation model

Different from common regression problems, the desired output value of the input data is difficult to determine for a RUL problem. It is usually impossible to evaluate the precise health condition and estimate the RUL of the system at each time step without an accurate physics based model. For this popular dataset, a piece-wise linear degradation model has been proposed in Ramasso (2014). The model assumes that the engines have a

Table 4

Allowed values for b per subset.

	FD001	FD002	FD003	FD004
b	30	20	30	18

constant RUL label in the early cycles, and then the RUL starts degrading linearly until it reaches 0 as shown in Fig. 3. The piecewise linear degradation assumption is used in this work. We denote the value of the RUL in the early cycles as R_e . Initially, R_e is randomly chosen between 90 and 140 cycles which is a reasonable range of values for this particular application. When the difference between the cycle count in the time window and the terminating cycle of the training data is less than the initial value of R_e , R_e begins the linear descent toward the terminating cycle.

3.3. Optimal data parameters

As mentioned in the previous sections the choice of the data-related parameters v has a large impact on the performance of the regressor. In this section, we present the framework for picking the optimal combination of the data-related parameters n_w , n_s and R_e while being computationally efficient.

Vector $v = (n_w, n_s, R_e)$ components specific to the C-MAPSS dataset are bounded such that $n_w \in [1, b]$, $n_s \in [1, 10]$, and $R_e \in [90, 140]$, where all the variables are integer. The value of b is different for different subsets of the data, Table 4 shows the different values of b for each subset.

Let $\mathbf{X}(v)$ be the training/cross-validate/test sets parametrized by v and used by the MLP to perform the RUL estimation. Finally, let $f(v) = e_{rms}(\mathbf{X}(v))$. Considering that $d = \hat{y} - y$ and that $\hat{y}$ depends on $\mathbf{X}(v)$, then one function evaluation of $f(v)$ requires training the MLP and computing the result of Eq. (1). Here we propose to optimize v such that

$$\min_{v \in \mathbb{Z}^3} f(v) \quad (4)$$

The problem to find optimal data-related parameters has no analytical descriptions. Therefore, no gradient information is available. An evolutionary algorithm is the natural choice for this optimization problem.

3.3.1. True optimal data parameters

The finite size of C-MAPSS dataset and finite search space of v allow an exhaustive search to be performed in order to find the true optimal data-related parameters. We would like to emphasize that although exhaustive search is possible for the C-MAPSS dataset, it is in no way a possibility in a more general setting. Nevertheless, the possibility to perform exhaustive search on the C-MAPSS dataset can be exploited to demonstrate the accuracy of the chosen EA and of the framework overall. In the following studies, we use the results and computational efforts

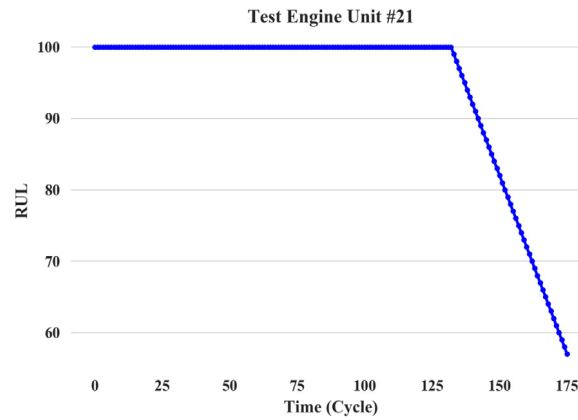


Fig. 3. Piecewise linear degradation for RUL.

Table 5

Exhaustive search results for subsets FD001 and FD02.

Dataset	argmin v	min $f(v)$	argmax v	max $f(v)$	Function evals.
FD001	[24, 1, 127]	15.11	[25, 10, 94]	85.19	8160
FD002	[16, 1, 138]	30.93	[17, 10, 99]	59.78	3060

of the exhaustive search as benchmarks to examine the accuracy and efficiency of the proposed approach.

We should note that the subsets of the data FD001 and FD003 have similar features and that the subsets FD002 and FD004 have similar features. Because of this, we have decided to just optimize the data-related parameters by considering the subsets FD001 and FD002 only. An exhaustive search is performed to find the true optimal values for v . The MLP is only trained for 20 epochs. Table 5 shows the optimal as well as the worst combinations of data-related parameters and the total number of function evaluations used by the exhaustive search. It is important to notice that for this experiment the window size is limited to be larger than or equal to 15.

Numerical experiments seem to suggest that, at least for CMAPSS dataset, the window size plays a big role in terms of the meaningful information used for prediction by the MLP. It also reflects the history-dependent nature of the aircraft engine degradation process. Furthermore, overlapping in the generated time windows seems to benefit the generated sequences of sensors.

3.3.2. Evolutionary algorithm for optimal data parameters

Evolutionary algorithms (EAs) are a family of methods for optimization problems. The methods do not make any assumptions about the problem, treating it as a black box that merely provides a measure of quality given a candidate solution. Furthermore, EAs do not require the gradient when searching for optimal solutions, making them very suitable for applications such as neural networks.

For the current application, the differential evolution (DE) method is chosen as the optimization algorithm (Storn & Price, 1997). Though other meta-heuristic algorithms may also be suitable for this application, the DE has established as one of the most reliable, robust and easy to use EAs. Furthermore, a ready to use Python implementation is available through the scipy package (Jones, Oliphant, Peterson, et al., 2001). Although the DE method does not have special operators for treating integer variables, a very simple modification to the algorithm, i.e. rounding every component of a candidate solution to its nearest integer, is used for this work.

As mentioned earlier, evolutionary algorithms such as the DE use several function evaluations when searching for the optimal

Table 6

Differential evolution hyper-parameters.

Population size	Generations	Strategy	MLP epochs
12	30	Best1Bin Engelbrecht (2007)	20

Table 7

Data-related parameters for each subset obtained with differential evolution.

Dataset	argmin v	min $f(v)$	Function evals.
FD001	[24, 1, 129]	15.24	372
FD002	[17, 1, 139]	30.95	372

solutions. It is important to consider that, for this application, one function evaluation requires retraining the neural network from scratch. This is not a desirable scenario, as obtaining the optimal data-related parameters would entail an extensive computational effort. Instead of running the DE for several iterations and with a large population size, we propose to run it just for 30 iterations, i.e. the generations in the literature of evolutionary computation, with a population size of 12, which seems reasonable given the size of the search space of v .

During the optimization, the MLP is trained for only 20 epochs. The small number of epochs of training the MLP is reasonable in this case because a small batch of data is used in the training, because we only look for the trend of the scoring indicators. Furthermore, it is common to observe that the parameters leading to lower score values in the early stages of the training are more likely to provide better performance after more epochs of trainings. The settings of the DE algorithm to find the optimal data-related parameters are listed in Table 6.

The optimal data-related parameters for the subsets FD001 and FD002 found by the DE algorithm are listed in Table 7. As can be observed, the results are in fact very close to the true optimal ones in Table 5 for both the subsets of the data. The computational effort is reduced by one order of magnitude when using the DE method as compared to the exhaustive search for the true optimal parameters. From the results in Table 7, it can be observed that the maximum allowable time window is always preferred while, on the other hand, small window strides yield better results. For the case of early RUL, it can be observed that larger values of R_e are favored.

3.4. The estimation algorithm

Having described the major building blocks of the proposed method, we now introduce the complete framework in the form of Algorithm 1.

Algorithm 1 ANN-EA RUL Estimation Framework

Input: Initial set of data-related parameters $v \in \mathbb{Z}^3$, Raw training/testing data $\mathbf{X} \in \mathbb{R}^{m \times s \times n_w}$, training labels $y \in \mathbb{Z}^m$ and number of training epochs for each evaluation of $f(v)$.

Output: Optimal set of data-related parameters $v^* \in \mathbb{Z}^3$

- 1: Choose regressor architecture (ANN, SVM, linear/logistic regression, etc).
- 2: Define $f(v)$ as in Section 3.3.
- 3: Optimize $f(v)$ using the preferred evolutionary algorithm, i.e. differential evolution, evolutionary strategies, genetic algorithm, etc, using the proposed guidelines from Section 3.3.2.
- 4: Use v^* to train the regressor for as many epochs as needed.

Table 8

Data-related parameters for each subset as obtained by DE.

Dataset	n_w	n_s	R_e	Input size (neurons)
FD001	24	1	129	336
FD002	17	1	139	238
FD003	24	1	129	336
FD004	17	1	139	238

Table 9

Scores for each dataset using the data-related parameters obtained by DE.

Data subset	RMSE				RHS			
	min	max	avg	STD	min	max	avg	STD
FD001	14.24	14.57	14.39	0.11	3.25	3.58	3.37	0.11
FD002	28.90	29.23	29.09	0.11	45.99	53.90	50.69	2.17
FD003	14.74	16.18	15.42	0.50	4.36	6.85	5.33	0.95
FD004	33.25	35.10	34.74	0.53	58.52	78.62	74.77	5.88

4. Evaluation of the proposed method

4.1. Experimental settings

In this section, we evaluate the performance of the proposed method. The architecture of the MLP is described in Table 3. We define an experiment as the process of training the MLP on any of the subsets (FD001 to FD004) and evaluate its performance using the subset's test set. The combinations of the optimal window size n_w , window stride n_s and early RUL R_e are presented in Table 8. We perform 10 different experiments for each data subset, the MLP is trained for 200 epochs using the train set for the corresponding data subset and evaluated using the subset's test set. The results for each dataset subset are averaged and presented in Table 9. Furthermore, the best model is saved and later used to generate the results presented in Section 4.2. All of the experiments were run using the Keras/Tensorflow framework, an NVIDIA GeForce 1080Ti GPU was used to speed up the training process.

4.2. Experimental results

The obtained results for $f(v)$ using the above setting are presented in Table 9. Notice that the performances obtained for datasets FD001 and FD002 are improved as compared with the results in Table 7. This is due to the fact that the MLP is trained for more epochs, thus obtaining better results.

We now compare the predicted RUL values vs the real RUL values for each of the datasets. For Figs. 4 to 7 we plot on the top subfigure the predicted RUL values (red lines) vs the real RUL values (green lines) while on the bottom subfigure we plot the error between real RUL and predicted RUL.

Fig. 4 shows the comparison for subset FD001, it can be observed that the predicted RUL values closely follow the real RUL

values with the exception of a pair of engines. The error remains small for most of the engines. Meanwhile, for FD002 it can be observed in Fig. 5 that the regressor overshoots several of the RUL predictions, especially in the positive spectrum. That is, the method predicts a RUL when in reality the real RUL is less than the predicted value. This is more evident in the second subplot where the maximum error is 138 at the magenta peak in the leftmost part of the plot.

Fig. 6 shows that for FD003 the predictions follow closely the real RUL values. The behavior for FD004 is similar to FD002 as depicted in Fig. 7, with most of the error such that the predictions of the RUL are larger than the real value.

Finally, Fig. 8 shows that most of the RUL predictions by the method are highly accurate. In the case of FD001, the predictions of 50% of the engines is smaller than 6 cycles. In the case of FD002, the predictions are acceptable with the error of first quartile being lower than 6 cycles and the error for 50% of the engines being less than 19 cycles. The cases for FD003 and FD004 are similar to FD001 and FD002, respectively.

Two conclusions can be drawn from the previous discussions. First, it can be observed that the number of operating conditions has a larger impact on the complexity of the data than the number of fault modes. This is because the subsets FD002 and FD004 exhibit larger errors than the subsets FD001 and FD003, although the general trend of prediction errors is similar among those two groups of datasets. Second, for the subsets FD002 and FD004, most of the error is related to the predictions larger than the real RUL values. Another observation is that larger window sizes usually lead to better predictions. This may be related to the history-dependent nature of the physical problem.

4.3. Comparison with other approaches

The performance of the proposed method is also compared against other state-of-the-art methods. The methods chosen for the comparison obey to two criteria: (1) that the method used is a machine learning (or statistical) method and (2) that the method is recent (6 years old at most). Most of the methods chosen here have only reported results on the test set FD001 in terms of e_{rms} . The results are shown in Table 10. The e_{rms} value of the proposed method in Table 10 is the mean value of 10 independent runs. The values of other methods are identical to those reported in their respective original papers.

From the comparison studies, we can conclude that the proposed method performs better than the majority of the chosen methods when taking into consideration the whole dataset FD001. Two existing methods come close to the performance of the proposed approach in this paper, namely the time window ANN (Lim et al., 2016) and the Networks Ensemble (Zhang et al., 2016). While the performance of these two methods comes close to the results of the proposed method in this paper, the proposed method is computationally more efficient. We believe that the use of the time window with a proper size makes the difference. Notice that three of the methods presented in Table 10 perform better than the proposed method, namely, the Convolutional Neural Network (Li et al., 2018), the RNN-Encoder-Decoder (Pankaj et al., 2016) and the modified ESN (Peng et al., 2012). Nevertheless in the case of Peng et al. (2012), the method can only predict 80 out of the 100 total engines while the deep CNN (Li et al., 2018) and RNN (Pankaj et al., 2016) approaches are much more computationally expensive than the MLP used in this work. In the case of the method proposed in Li et al. (2018), the neural network model has four layers of convolutions and two more layers of fully connected layers. On the other hand, the RNN-Encoder-Decoder (Pankaj et al., 2016) makes use of a much more complicated scheme of two RNNs, one for encoding the

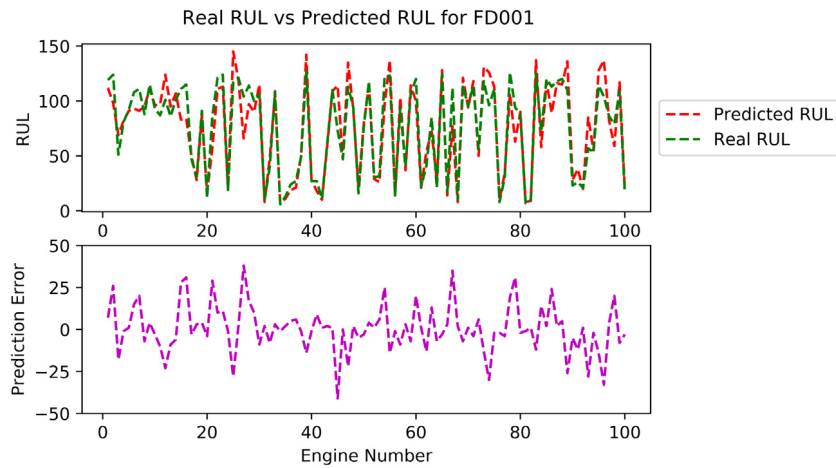


Fig. 4. Comparison of predicted RUL values vs real RUL values for dataset FD001.

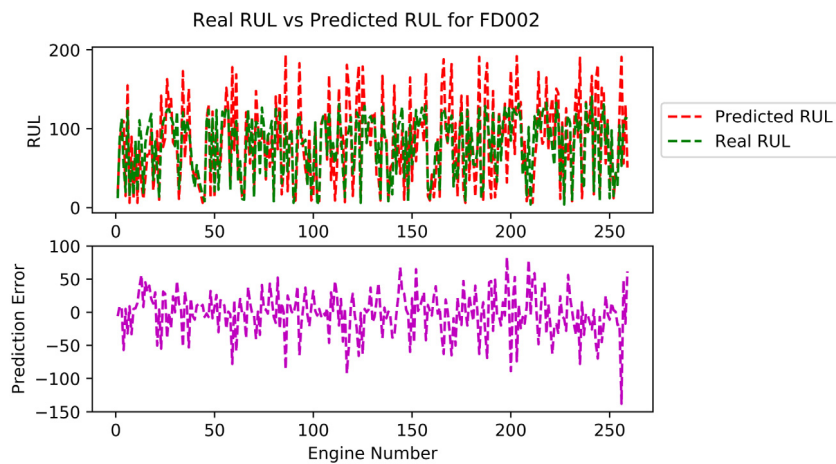


Fig. 5. Comparison of predicted RUL values vs real RUL values for dataset FD002.

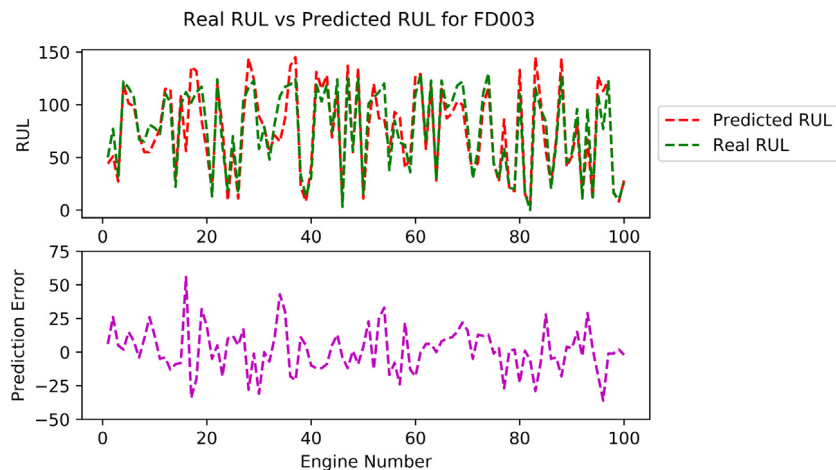


Fig. 6. Comparison of predicted RUL values vs real RUL values for dataset FD003.

sequences and one for decoding them. While specialized libraries such as TensorFlow or Keras make RNNs easier to use, they still remain up to date as some of the most computationally expensive architecture to train given their sequential nature. Finally, we would like to emphasize that the used MLP for this approach is

one of the simplest ones in the reviewed literature. Furthermore, the framework proposed is simple to understand and implement, robust, generic and light-weight. These are the features important to highlight when comparing the proposed method against other state-of-the-art approaches.

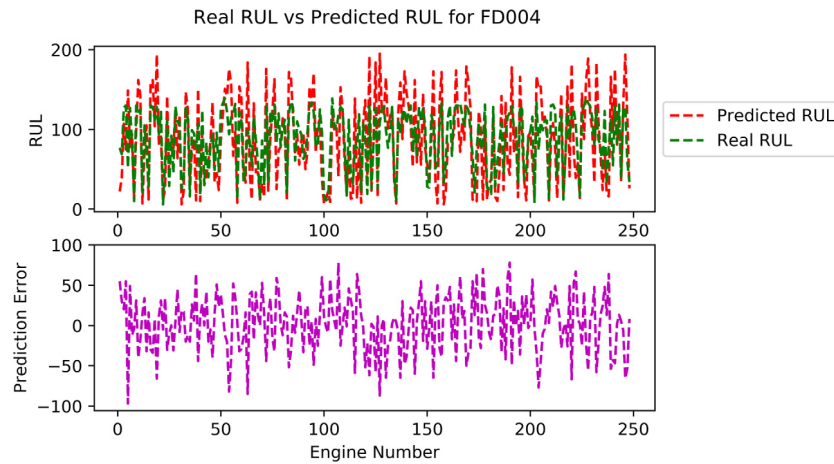


Fig. 7. Comparison of predicted RUL values vs real RUL values for dataset FD004.

Table 10

Performance comparison of the proposed method and the latest related papers on the C-MAPSS dataset.

Method	e_{rms}
ESN trained by Kalman Filter (Peng, Wang, Wang, Liu, & Peng, 2012)	63.45
Support Vector Machine Classifier (Louen, Ding, & Kandler, 2013)	29.82
Time Window Neural Network (Lim et al., 2016)	15.16
Multi-objective deep belief networks ensemble (Zhang, Lim, Qin, & Tan, 2016)	15.04
Deep Convolutional Neural Network (Babu, Zhao, & Li, 2016)	18.45
Proposed method with $n_w = 30$, $n_s = 1$ and $R_e = 128$	14.39
Modified approach with classified sub-models of the ESN (Peng et al., 2012).	7.021
Only 80 of 100 engines predicted	
Deep CNN with time window (Li et al., 2018).	13.32
RNN-Encoder-Decoder (Pankaj et al., 2016).	12.93

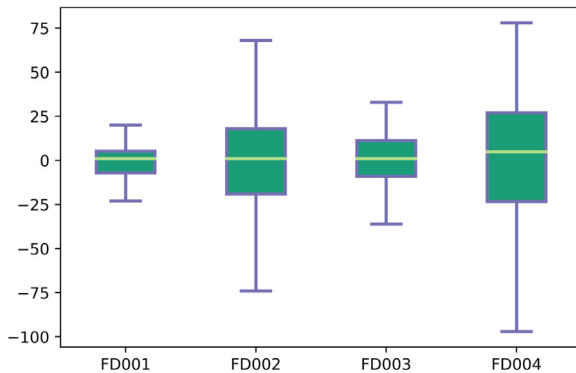


Fig. 8. Prediction error of the MLP for each dataset.

5. Conclusions

We have presented a novel framework for predicting the RUL of mechanical components. While the method has been tested on the jet-engine dataset C-MAPSS, the method is general enough such that it can be applied to other similar systems. The framework makes use of a strided moving time window to generate the training and test records. A shallow MLP to make the predictions of the RUL has been found to be sufficient for the C-MAPSS dataset. The evolutionary algorithm DE needs to be run just once to find the best data-related parameters that optimize the scoring functions. The resulting model of the application of the framework presented in this paper demonstrated to be accurate and computationally efficient, specially when applied to large datasets in real applications. The compactness of the resulting

Table A.1

Proposed neural network architecture 1.

Layer	Neurons	Activation	Additional information
Fully connected	20	ReLU	L1 = 0.1, L2 = 0.2
Fully connected	20	ReLU	L1 = 0.1, L2 = 0.2
Fully connected	1	Linear	L1 = 0.1, L2 = 0.2

Table A.2

Proposed neural network architecture 2.

Layer	Neurons	Activation	Additional information
Fully connected	50	ReLU	L1 = 0.1, L2 = 0.2
Fully connected	20	ReLU	L1 = 0.1, L2 = 0.2
Fully connected	1	Linear	L1 = 0.1, L2 = 0.2

model makes it suitable for applications that have limited computational resources such as embedded systems. It is important to note that the framework itself does involve some computations, nevertheless such computations are done off-line. Furthermore, the comparison with other state-of-the-art methods has shown that the proposed method is the best overall performer.

Two major features of the proposed framework are its generality and scalability. While for this study, specific regressors and evolutionary algorithms are chosen, many other combinations are possible and may be more suitable for different applications. Furthermore, the framework can, in principle, be used for model-construction, i.e. generating the best possible neural network architecture tailored to a specific application. Thus, future work will consider extensions to the framework to make it applicable to tasks such as model selection. An analysis of the influence of the window stride parameter n_s will also be considered for future work.

Table A.3

Proposed neural network architecture 3.

Layer	Neurons	Activation	Additional information
Fully connected	100	ReLU	L1 = 0.1, L2 = 0.2
Fully connected	50	ReLU	L1 = 0.1, L2 = 0.2
Fully connected	1	Linear	L1 = 0.1, L2 = 0.2

Table A.4

Proposed neural network architecture 4.

Layer	Neurons	Activation	Additional information
Fully connected	250	ReLU	L1 = 0.1, L2 = 0.2
Fully connected	50	ReLU	L1 = 0.1, L2 = 0.2
Fully connected	1	Linear	L1 = 0.1, L2 = 0.2

Table A.5

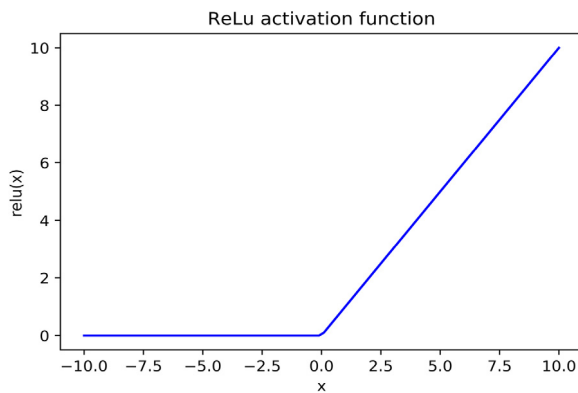
Proposed neural network architecture 5.

Layer	Neurons	Activation	Additional information
Fully connected	20	ReLU	L1 = 0.1, L2 = 0.2
Fully connected	1	Linear	L1 = 0.1, L2 = 0.2

Table A.6

Proposed neural network architecture 6.

Layer	Neurons	Activation	Additional information
Fully connected	10	ReLU	L1 = 0.1, L2 = 0.2
Fully connected	1	Linear	L1 = 0.1, L2 = 0.2

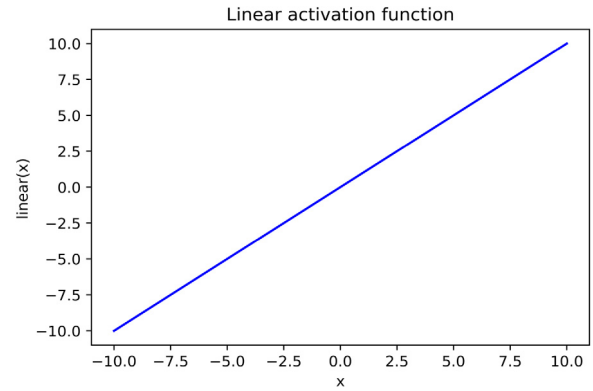
**Fig. B.9.** ReLU activation function.

Acknowledgments

The authors acknowledge the funding from Conacyt Project No. 285599 and a grant (11572215) from the National Natural Science Foundation of China.

Appendix A. Tested neural network architectures

In this appendix we present the tested neural network architectures. Each table represents a neural network model. Each row in the table represents a neural network layer while each column describes each one of the key parameters of the layer such as the type of layer, number of neurons in the layer, activation function of the layer and whether regularization is used, where L1 denotes the L1 regularization factor and L2 denotes the L2 regularization factor, the order in which the layers are appended from the table is top–bottom (see Tables A.1–A.6).

**Fig. B.10.** Linear activation function.

Appendix B. Activation functions definitions

Here we define some of the activation functions used in our neural network models. For the following definitions assume a vector $x \in \mathbb{R}^n$ (see Figs. B.9 and B.10).

B.1. ReLU activation function

$$f(x) = \max(0, x)$$

B.2. Linear activation function

$$f(x) = x$$

References

- Babu, G. S., Zhao, P., & Li, X. (2016). Deep convolutional neural network based regression approach for estimation of remaining useful life. In *The 21st international conference on database systems for advanced applications*, (pp. 214–228). Springer International Publishing.
- Benkedjouh, T., Medjaher, K., Zerhouni, N., & Rechak, S. (2013). Remaining useful life estimation based on nonlinear feature reduction and support vector regression. *Engineering Applications of Artificial Intelligence*, 26(7), 1751–1760.
- Bühlmann, P., & Geier, S. (2011). *Statistics for high dimensional data. Methods, theory and applications*. Springer.
- Dong, M., & He, D. (2007). A segmental hidden semi-Markov model (HSMM)-based diagnostics and prognostics framework and methodology. *Mechanical Systems and Signal Processing*, 21(5), 2248–2266.
- Engelbrecht, D. P. (2007). *Computational intelligence. An introduction*. Wiley.
- Gebräuel, N. Z., Lawley, M. A., Li, R., & Parmeshwaran, V. (2004). Residual-life distributions from component degradation signals: A neural-network approach. *IEEE Transactions on Industrial Electronics*, 51(3), 150–172.
- Gebräuel, N. Z., Lawley, M. A., Liu, R., & Ryan, J. K. (2005). Residual-life distributions from component degradation signals: a Bayesian approach. *IEEE Transactions*, 37(6), 543–557.
- Jones, E., Oliphant, T., Peterson, P., et al. (2001). SciPy: Open source scientific tools for Python. URL <http://www.scipy.org/>. [Online; Accessed 06/2018].
- Kong, C., & Lucey, S. (2017). Take it in your stride: Do we need striding in CNNs?. *Computing Research Repository*, abs/1712.02502.
- Laredo, D., Chen, X., Schütze, O., & Sun, J. Q. (2018). ANN-EA for RUL estimation, source code. [Online]. URL https://github.com/dlaredo/NASA_RUL_CMPS-.
- Lee, J., Wu, F., Zhao, W., Ghaffari, M., Liao, L., & Siegel, D. (2014). Prognostics and health management design for rotary machinery systems - Reviews, methodology and applications. *Mechanical Systems and Signal Processing*, 42(12), 314–334.
- Li, X., Ding, Q., & Sun, J. Q. (2018). Remaining useful life estimation in prognostics using deep convolution neural networks. *Reliability Engineering & System Safety*, 172, 1–11.
- Lim, P., Goh, C. K., & Tan, K. C. (2016). A time window neural networks based framework for remaining useful life estimation. In *Proceedings international joint conference on neural networks* (pp. 1746–1753).

- Liu, J., & Wang, G. (2009). A multi-state predictor with a variable input pattern for system state forecasting. *Mechanical Systems and Signal Processing*, 23(5), 1586–1599.
- Liu, J., Wang, M., & Yang, Y. (2012). A data-model-fusion prognostic framework for dynamic system state forecasting. *Engineering Applications of Artificial Intelligence*, 25(4), 814–823.
- Louen, C., Ding, S. X., & Kandler, C. (2013). A new framework for remaining useful life estimation using support vector machine classifier. In *Conference on control and fault-tolerant systems* (pp. 228–233).
- Mosallam, A., Medjaher, K., & Zerhouni, N. (2013). Nonparametric time series modelling for industrial prognostics and health management. *International Journal of Advanced Manufacturing Technology*, 69(5), 1685–1699.
- Pankaj, M., Vishnu, T., Anusha, R., Gaurangi, A., Lovekesh, V., Puneet, A., et al. (2016). Multi-sensor prognostics using an unsupervised health index based on LSTM encoder-decoder. *Computing Research Repository*, arXiv:1608.06154.
- Pecht, M., & Jaai (2010). A prognostics and health management roadmap for information and electronics rich-systems. *Microelectronics Reliability*, 50(3), 317–323.
- Peng, Y., Wang, H., Wang, J., Liu, D., & Peng, X. (2012). A modified echo state network based remaining useful life estimation approach. In *IEEE conference on prognostics and health management* (pp. 1–7).
- Qian, Y., Yan, R., & Gao, R. X. (2017). A multi-time scale approach to remaining useful life prediction in rolling bearing. *Mechanical Systems and Signal Processing*, 83, 549–567.
- Ramasso, E. (2014). Investigating computational geometry for failure prognostics. *International Journal of Prognostics and Health Management*, 5(1), 1–18.
- Saxena, A., & Goebel, K. (2008). PHM08 challenge data set. [Online] Available at: <https://ti.arc.nasa.gov/tech/dash/groups/pcoe/prognostic-data-repository/>.
- Saxena, A., Goebel, K., Simon, D., & Eklund, N. (2008). Damage propagation modeling for aircraft engine run-to-failure simulation. In *International conference on prognostics and health management* (pp. 1–9). IEEE.
- Sikorska, J. Z., Hodkiewicz, M., & Ma, L. (2011). Prognostic modelling options for remaining useful life estimation by industry. *Mechanical Systems and Signal Processing*, 25(5), 1803–1836.
- Storn, R., & Price, K. (1997). Differential evolution – A simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4), 341–359. <http://dx.doi.org/10.1023/A:1008202821328>.
- Yu, W., & Kuffi, H. (2001). A new stress-based fatigue life model for ball bearings. *Tribology Transactions*, 44(1), 11–18.
- Zaidan, M., Mills, A., & Harrison, R. (2013). Bayesian Framework for aerospace gas turbine engine prognostics. In *Aerospace conference* (pp. 1–8). IEEE.
- Zhang, C., Lim, P., Qin, A., & Tan, K. (2016). Multiobjective deep belief networks ensemble for remaining useful life estimation in prognostics. *IEEE Transactions on Neural Networks and Learning Systems*, 99, 1–13.
- Zhao, Z., Bin, L., Wang, X., & Lu, W. (2017). Remaining useful life prediction of aircraft engine based on degradation pattern learning. *Reliability Engineering & System Safety*, 164, 74–83.